

Market Data Parsing on FPGA NIC

Motivation

The Latency Problem in HFT

High-frequency trading systems rely on low latency processing of market data where microseconds matter. Traditional software networking stacks introduce unpredictable latency through kernel interrupts, memory copies, and OS scheduling. By moving packet parsing directly into the FPGA, decisions happen at the clock cycle a packet arrives with no CPU involvement on the critical path.

Instead of waiting for packets to travel through the operating system into a software application, the FPGA can inspect packets immediately as they arrive from the network interface. This allows market data messages to be identified and decoded with significantly lower and more predictable latency.

System Architecture

The system is built on top of the AMD/Xilinx OpenNIC shell running on an Alveo U55C FPGA. OpenNIC handles the low-level networking stack, FPGA interface management, and packet transport between the host machine and the FPGA logic. The full data path is:

- **100GbE Network** (physical cable) to CMAc Subsystem (100GbE MAC, physical layer)
- **AXI-Stream 512-bit bus** allows 64 byte “beat” per clock cycle at 250 MHz. The AXI-Stream bus is how data flows from the network into the custom parsing logic.
- **packet_parser module** performs header extraction and IEX DEEP decoding (parsing logic goes in the box_250mhz plugin slot of the OpenNIC shell).
- **QDMA / PCIe** lets the DMA engine blasts parsed packets into host RAM over PCIe Gen4. CPU receives notice only after data is already in memory.
- **Zero CPU on critical path:** Filtered packets are dropped before QDMA so they never consume PCIe bandwidth or CPU cycles.

The custom parser operates as a passthrough module by default meaning packets continue propagating through the OpenNIC data path unchanged while protocol fields are simultaneously decoded in parallel hardware logic.

IEX DEEP Protocol

IEX DEEP (Deep Book) is the real-time market data feed published by the Investors Exchange (IEX), carrying limit order book updates for all listed securities. IEX DEEP was chosen because it publishes T+1 packet captures publicly so this enabled realistic offline verification without requiring a live market connection.

IEX DEEP market data is transmitted over UDP multicast using the IEX-TP framing protocol. The UDP payload offset was calculated based on a 14-byte Ethernet header, 20-byte IPv4 header, and 8-byte UDP header, placing the UDP payload at byte offset 42.

IEX DEEP Message Parsing Pipeline

The parser and decoder together form a layered streaming pipeline with all combinational logic and zero additional latency:

- **Header parsing:** The parser extracts the message length and type from the packet headers.

- **1024-bit window:** A 1024-bit window formed from the current and previous 512-bit AXI beats, enables correct processing of messages that span beat boundaries.
- **Length array:** Message start and end offsets are generated every cycle to identify individual messages within the data stream.
- **Price Level Update (PLU) filter:** Only buy-side (0x38) and sell-side (0x35) PLU messages are retained while all other message types are discarded.
- **FIFO output:** The decoder extracts the symbol, price, size, and side information and stores the results in a FIFO buffer to prevent message loss during periods of high traffic.
- **Multi-beat crossover:** The previous AXI beat is stored in a register and combined with the current beat to form a 1024-bit window, ensuring that messages crossing AXI beat boundaries remain contiguous and are processed correctly.

Price Level Update Message

DEEP broadcasts a real-time PLU Message each time a displayed price level on IEX is updated during the trading day. When a price level is removed, IEX broadcasts a size of zero (0x0) for that level.

Field Name	Offset	Length	Type	Description/Notes
Message Type	0	1	Byte	'8' (0x38) – Buy Side PLU / '5' (0x35) – Sell Side PLU
Event Flags	1	1	Byte	Identifies event processing by the System
Timestamp	2	8	Timestamp	Time stamp of the price level update
Symbol	10	8	String	Security identifier
Size	18	4	Integer	Aggregate quoted size
Price	22	8	Price	Price level to add/update in the IEX Order Book

Hardware

AXI-Stream Beat Layout

We used a 512-bit AXI-Stream beat, so there are 64 bytes per clock cycle at 250 MHz. Entire Ethernet + IP + TCP/UDP header fits in the first beat. All fields extracted in a single always_comb block with zero additional clock cycles.

- **Ethernet (bytes 0-13):** dst MAC [47:0], src MAC [95:48], EtherType [111:96]
- **IPv4 header (bytes 14-33):** protocol [191:184], src IP [239:208], dst IP [271:240]
- **TCP/UDP (bytes 34-41):** src port [287:272], dst port [303:288]
- **IEX payload (bytes 42-63):** msg type [336+:8], symbol, price, size

OpenNIC Integration

- Instantiated inside box_250mhz via user plugin include file
- Intermediate wires carry parsed output to p2p plugin
- AXI backpressure preserved via tready/tvalid handshake
- Packet drop via m_axis_tvalid suppression for zero latency

Key Source Files

- **packet_parser.sv:** Main AXI-stream parser. Extracts Ethernet/IP/UDP fields, performs IEX-TP detection, packet classification, and forwards packets through the OpenNIC datapath.

- **deep_decode.sv:** Streaming DEEP market-data decoder. Parses DEEP message structures, filters PLU updates, extracts fields (price/size/symbol), and handles cross-beat message alignment using a 1024-bit sliding window.
- **tb_packet_parser.sv:** SystemVerilog testbench for simulation and validation using synthetic Ethernet/IP/UDP/IEX packets.
- **user_plugin_250mhz_inst.vh:** This is the OpenNIC integration wrapper, it connects packet_parser.sv into the RX AXI-stream datapath inside the 250 MHz OpenNIC plugin region.

Verification

Simulation Testbench

To accelerate development and avoid unnecessary FPGA recompilation cycles, simulation infrastructure was developed using a custom SystemVerilog testbench (tb_packet_parser.sv). The testbench manually constructs synthetic Ethernet frames at the bit level and injects them into the AXI-Stream interface.

Test packets included:

- IPv4/TCP packets
- IPv4/UDP packets
- ARP packets
- Synthetic IEX DEEP packets with IEX-TP protocol ID 0x8004 and channel ID 1

Simulation validated:

- Protocol classification correctness
- Field extraction offsets and packet-aware counting behavior
- DEEP candidate identification and PLU message filtering
- Cross-beat message handling via 1024-bit sliding window
- Symbol, price, and size extraction from target messages

Live Traffic Test

Live network communication was validated between HFT server interfaces using the ens1f1 interface. ICMP ping traffic was sent to target IP 10.30.203.56, confirming:

- 0% packet loss
- Sub-millisecond latency on real 100GbE hardware
- Custom parser logic did not disrupt packet forwarding through the OpenNIC data path

Results

1. **Setup & Bitstream:** OpenNIC bitstream generated on the fintech lab's hft06 server and programmed onto Alveo U55C on the fintech lab's hft01 server. We brought up the FPGA-backed Linux network interfaces through PCIe.
2. **Packet Parsing Logic:** Custom SystemVerilog parser extracts all Ethernet, IP, TCP, and UDP header fields in a single clock cycle at 250 MHz.
3. **DEEP Protocol Decoding:** IEX DEEP price level update messages (0x38/0x35) decoded in simulation using T+1 packet captures. Symbol, price, size, and side extracted.
4. **IPv4/TCP & IPv4/UDP Verification:** Both packet types verified in simulation. Live ping over ens1f1 confirmed 0% packet loss and sub-ms latency on real hardware.

Future Work

1. **Expanded Protocol Support:** The parser could be extended to support additional exchange formats and message types, enabling broader real-time market data processing directly on the FPGA. Planned expansion includes NASDAQ ITCH and NYSE PILLAR to build a general multi-feed market data engine.
2. **Hardware Monitoring and Observability:** Performance counters, drop tracking, and latency histograms could be added to enable detailed real-time monitoring of system behavior during live traffic.
3. **Real Time Analytics:** The parser could be extended to compute live market features such as spread, depth, and trade statistics directly in hardware.
4. **Live FPGA Traffic:** Send real packets through the FPGA data path and verify parser sees live traffic.
5. **Full DEEP Decoder:** Parse actual DEEP message types and extract symbol, price, size, and timestamp.
6. **Order Book Reconstruction:** Maintain bid/ask levels in FPGA BRAM and update book from DEEP messages.
7. **FPGA Analytics:** Compute spread, midprice, and imbalance and generate market signals in hardware.
8. **Benchmarking:** Measure FPGA vs. software latency and compare throughput and CPU usage.